(3 Shifts) Hybrid Puzzle Scheduling Heuristic

October 13, 2026

1 Introduction

This document describes a hybrid (Exact and Greedy) heuristic for yearly employee shift scheduling. The problem involves assigning employees to shifts across a full calendar year, respecting a set of hard and soft constraints (staffing minimums and ideals, maximum consecutive working days, maximum annual working days, Sunday/holiday caps, team eligibility, and shift-transition minimum rest).

The approach adopted here decomposes the year into fixed-length periods (periods of η days), solving a smaller, tractable ILP for each period, and complementing it with a set of greedy post-processing phases that repair local imbalances the weekly ILPs cannot see.

Sundays, holidays, and vacation days are deliberately excluded from the weekly pattern-generation and ILP phase. These days are hard to keep tracking in the ILP solver. So, instead, they are handled separately in a dedicated final phase, once the regular weekday structure has already been fixed.

The heuristic operates in 4 sequential phases:

1. **Phase 1 (Weekly ILP pattern assignment)**: generates all valid weekly work patterns and solves an ILP that selects, for each employee, the pattern and team minimizing the shortage of minimum and ideal staffing requirements;
2. **Phase 2 (Remove excessive ideals)**: removes employees assigned in excess of the ideal requirement for a given day/shift/team;
3. **Phase 3 (Assign Sundays and holidays)**: a greedy heuristic that assigns employees to Sunday and holiday shifts, respecting the $H_{\max}$ cap.
4. **Phase 4 (Fix Ideals on Util Days)**: a greedy heuristic, driven by the scoring function in Section 6.1, that fills remaining weekday deficits left unresolved by Phase 1;

Phases 3 and 4 rely on the local scoring function described in Section 6.1, while Phase 1 is formulated as an Integer Linear Program solved with the CBC solver via PuLP, as detailed in Appendix A.

2 Indexes

Index	Description
E	Set of Employees
D	Set of days, $D = \{1, 2, \ldots, \lvert D\rvert\}$, where $\lvert D\rvert = \begin{cases} 365, & \text{if common year} \\ 366, & \text{if leap year} \end{cases}$
S	Set of Shifts, $S = \{1, 2, 3\}$ (Morning, Afternoon, Night)
T	Set of Teams
T_e	Set of teams t that employee can work
T^*	Set of Teams order by number of Shifts s in decrescent order
$R \subseteq E$	Set of Employees ranked by Score
$SH \subset D$	Set of Sunday and Holiday days
$L \subset D$	Set of deficit days (i.e, days covered less than the ideal), $L = [(d_1, s_1, t_1, \gamma_1), (d_2, s_2, t_2, \gamma_2), ..]$
W	Set of Weeks (Length defined by spacing η parameter; Each week $w \in W \subseteq D$)

All Sets with $\sim$ above means that is randomly ordered (Eg. $\tilde{\text{D}}$ -Set of Random days)

3 Parameters

Parameter	Description
θ_{dst}	Minimum number of employees required on day d, shift s, team t
β_{dst}	Ideal number of employees on day d, shift s, team t
α_{ed}	1 if employee e worked on day d, 0 otherwise
η	Predefined Week length
τ	Target number of working days
$D_{\max}$	Maximum working days per year per employee (223)
$H_{\max}$	Maximum Sundays/holidays worked per year per employee (22)
$C_{\max}$	Maximum consecutive working days (5)
W_{pace}	Weight for pace component in scoring function (0.60)
W_{seq}	Weight for streak component in scoring function (0.25)
W_{trans}	Weight for shift-transition component in scoring function (0.10)
W_{teams}	Weight for team flexibility component in scoring function (0.03)
K	Cap Value for patterns selection (limits the number of patterns per employee for a certain spacing η)

4 State Variables

Variable	Description
TotalDays_e	Number of working days assigned to employee e so far
Streak_e	Current consecutive working-day streak for employee e
PrevShift_{ed}	Shift assigned to employee e on the previous calendar day $d-1$
NextShift_{ed}	Shift assigned to employee e on the next calendar day $d+1$
Score_e	Priority score of employee e, updated after each day
Assignment	Dictionary mapping each employee e to a list of tuples (d, s, t)
Cov_{dst}	Current coverage for day d, shift s and team t
γ	Deficit, i.e (ideal value - actual value)

5 Rules and Constraints Covered

The heuristic enforces the hard constraints below and aims at covering the minimum staffing requirements (soft constraint)::

- Full coverage of minimum staffing requirements per shift and team (*soft constraint*);
- At most one shift assignment per employee per day (*Hard Constraint*);
- At most $C_{\max} = 5$ consecutive working days per employee (*Hard Constraint*);
- At most $D_{\max} = 223$ working days per year per employee (*Hard Constraint*);
- At most $H_{\max} = 22$ Sundays/holidays worked per year per employee (*Hard Constraint*);
- Employee vacation periods V_e must not be violated (*Hard Constraint*);
- The **ForbiddenTransitions** between assigned shifts are as follows:
  - Night → Morning
  - Night → Afternoon
  - Afternoon → Morning
- Each employee must be assigned only to allowed teams (*Hard Constraint*);

6 Auxiliary Functions of the Heuristic

6.1 Employee Scoring Function

The scoring function $\text{Score}(e, d)$ assigns a priority value to each employee e at the end of day d based on his assignments and the value is applied in next day. Higher scores indicate higher assignment priority. The score combines four components:

$$\text{Score}(e, d) = W_{\text{pace}} \cdot \phi_{\text{pace}}(e) + W_{\text{seq}} \cdot \phi_{\text{seq}}(e) + W_{\text{trans}} \cdot \phi_{\text{trans}}(e) + W_{\text{teams}} \cdot \phi_{\text{teams}}(e) \quad (1)$$

Pace component ϕ_{pace} Measures how far the employee is from the expected annual workload. Employees who have worked fewer days than expected receive a higher score:

$$\phi_{\text{pace}}(e) = \left(1 - \left(\frac{\text{TotalsDays}_e}{D_{\max}}\right)\right)^2 \quad (2)$$

Streak component ϕ_{seq} Penalises employees who are close to the maximum consecutive-day limit, favouring those with shorter current streaks:

$$\phi_{\text{seq}}(e) = \left(1 - \left(\frac{\text{Streak}_e}{C_{\max}}\right)\right)^2 \quad (3)$$

Transition component ϕ_{trans} Rewards employees whose previous shift creates a favourable transition context for the next assignment. Employees who worked a Night shift (shift 3) or an Afternoon shift (shift 2) the previous day receive a higher value, since those shifts are more constrained in what can follow them:

$$\phi_{\text{trans}}(e) = \begin{cases} 0.75 & \text{if PrevShift}_{ed} = 3 \\ 0.5 & \text{if PrevShift}_{ed} = 2 \\ 0.0 & \text{otherwise} \end{cases} \quad (4)$$

Team flexibility component ϕ_{teams} Gives a small bonus to employees who are qualified in fewer teams, so that employees with rare qualifications are assigned first:

$$\phi_{\text{teams}}(e) = \left(1 - \frac{|T_e|}{|T|}\right) \quad (5)$$

6.2 Global Assignment Order

At the start of each day, employees are ranked in decreasing order of $\text{Score}(e, d)$. Ties are broken by random shuffling (performed before the sort), ensuring fairness and diversity across heuristic runs.

6.3 Shift-Transition Validation

Before assigning shift s to employee e on day d, the heuristic checks that the transition from the previously worked shift ($s' = \text{PrevShift}_{ed}$) is valid. A transition (s, s') is valid if and only if

$(s, s') \notin$ ForbiddenTransitions. This enforces a minimum rest interval between consecutive work days.

In Phase 3 and Phase 4, transition validation is applied *bidirectionally*: a candidate shift s on day d must be simultaneously compatible with the shift already assigned on day $d-1$ (backward check) and with the shift already assigned on day $d+1$, if any (forward check). Formally, s is admissible if and only if:

$$(\text{PrevShift}_{ed}, s) \notin \text{ForbiddenTransitions} \quad \text{and} \quad (s,\ \text{NextShift}_{ed}) \notin \text{ForbiddenTransitions} \tag{6}$$

6.4 Consecutive Days Check

This function verifies that adding employee e on day d would not create a consecutive streak exceeding $C_{\max}$. It counts the backward streak $b(e, d)$ (consecutive days worked immediately before d) and the forward streak $a(e, d)$ (consecutive days already assigned immediately after d):

$$\text{TotalStreak}(e, d) = b(e, d) + 1 + a(e, d) \tag{7}$$

The assignment is only allowed if $\text{TotalStreak}(e, d) \leq C_{\max}$. This two-sided check prevents hidden violations that arise when a new assignment fills a gap between two existing runs.

6.5 Assign Function

The `_assign` function is responsible for assigning an employee to a specific shift, team, and day. This operation is performed atomically, ensuring that all data structures used by the system remain consistent after each assignment.

The use of this method centralizes all update operations on the system's data structures, reducing the risk of inconsistencies resulting from direct modifications to internal records.

6.6 Remove Function

The `_unassign` function performs the inverse operation of the `_assign` function, removing a previously assigned shift from an employee. As with assignment, the removal is carried out atomically, ensuring the consistency of all data structures used by the system.

To prevent negative values in the counters, the function guarantees that they never fall below zero. In this way, the removal of assignments preserves data integrity and ensures the correct accounting of each employee's workload.

6.7 Pattern Generation

The `build_Weekly_Possible_Attributions` function generates all valid shift sequences for a planning horizon of η days. Each sequence represents a possible daily work assignment for one employee over one period, where the value `0` denotes a day off and values `1` to $|S|$ denote the available shift types.

Sequences are constructed via a *backtracking* algorithm that explores all combinations of rest and work days recursively. The following hard constraints are enforced *during* construction (pruning):

R1 - The number of consecutive working days may never exceed $C_{\max}$; any branch that reaches this limit is immediately pruned.

R2 - Every transition between two adjacent working shifts must satisfy the `existing constraint`, which forbids invalid rotations (e.g. Night → Morning).

R3 - A lower-bound pruning rule discards branches where the number of remaining days is insufficient to reach `sigma_min`, the minimum number of working days required for the period. An upper-bound pruning rule discards branches once the number of working days already assigned reaches `sigma_max`, the maximum allowed.

The bounds `sigma_min` and `sigma_max` are derived from the target number of working days per period, τ, defined as

$$\tau \;=\; \frac{201}{|D| \,/\, \eta},$$

and a context-sensitive margin m:

$$\texttt{sigma_min} = \lfloor \tau \rceil - m, \qquad \texttt{sigma_max} = \lfloor \tau \rceil + m$$

where $\lfloor \tau \rceil$ denotes rounding to the nearest integer and $m = 1$.

The numerator 201 is the target number of annual working days on regular days, i.e. $D_{\max} - H_{\max} = 223 - 22$, since Sundays and holidays are handled separately in Phase 3.

After a complete sequence has been generated, one additional structural filter is applied:

R4 - Sequences belonging to full periods that contain three or more disjoint work blocks, depending on η, are discarded, as highly fragmented patterns of moderate length are rarely selected by the ILP and add unnecessary search space.

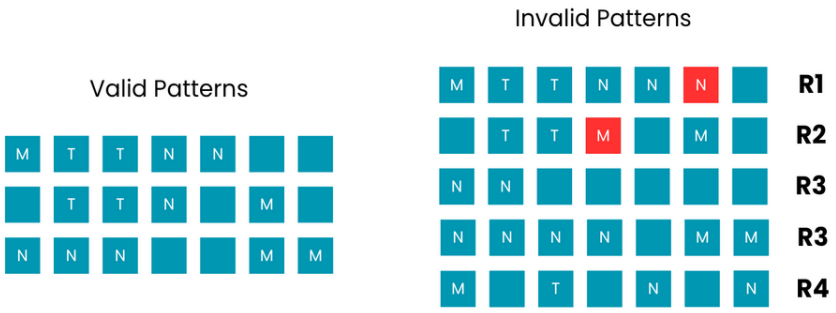


Figure 1: Patterns Generation Examples

6.8 The `Pontuate` Function

The `Pontuate` function is the central decision engine of the weekly scheduling loop. Its purpose is to select, for each employee, the shift pattern that best covers the staffing requirements for the current period while respecting three classes of individual constraints: the shift sequence count inherited from the previous period, the employee's vacation days within the current period, and the minimum rest rules governing transitions between consecutive shift types. The output is an Integer Linear Programme (ILP) whose solution assigns exactly one admissible pattern and one permitted team to each employee. The ILP formulation is presented in full in Appendix A.

Pre-processing: building the admissible pattern set

Before the ILP is constructed, the full catalogue of candidate patterns generated before is reduced, for each employee individually, to an *admissible set* $\mathcal{P}_e$ that is guaranteed to be feasible given that employee's personal circumstances in the current period. The reduction proceeds in three stages.

1. Inter-period transition constraints. The pattern assigned during the immediately preceding period imposes two restrictions on how the current period may begin, both derived from the principle that work schedules must guarantee adequate rest across period boundaries.

The first restriction concerns the *trailing streak*: the number of consecutive working days at the tail of the previous pattern. If the previous period ended with k consecutive working days, the employee may work at most $C_{\max} - k$ days at the start of the current period before a mandatory rest day, where $C_{\max}$ is the maximum number of consecutive working days permitted. Any pattern whose opening block of non-zero shifts exceeds this budget is removed.

The second restriction concerns the *type of the last shift* worked. Because descending transitions between shift types are inadmissible. Specifically : Afternoon $\rightarrow$ Morning and Night $\rightarrow$ {Morning, Afternoon}.

2. Masking. Every pattern that survives the transition filters is then *masked*: the shift value at each closed offset (Sundays, public holidays, and vacation days) is replaced by zero, yielding the pattern as it will actually appear in the employee's schedule.

3. Catalogue cap. Even after the filters above, the admissible set may remain large for longer planning horizons, inflating the ILP without a proportionate gain in solution quality. A period-specific cap K is therefore enforced. When the admissible set exceeds this limit, patterns are sorted by proximity to the target τ and only the first K are kept. The cap values used in practice are reported in Table 1 and were selected as described below.

Period length η (days)	Catalogue cap K
2	7
3	26
4	18
5	201
6	77
7	139
8	335
9	694

Table 1: Maximum number of admissible patterns K retained per employee for each supported period length.

Selecting the cap values

The cap cannot be arbitrary. If it is too small, the patterns needed to cover the minimum staffing requirements are discarded and the solution deteriorates; if it is too large, the catalogue, and therefore the ILP time, grows exponentially with η. For each period length, the cap was set to the **smallest value found that still reaches the target number of minimum shortages**, taken from the full-year exact ILP, described in the companion report *Heuristic Results (3 Shifts)*. The cap is tuned to preserve the *Minimums* metric and not to optimise the *Ideals* metric.

Acceptance criterion. The heuristic is stochastic (random tie-breaking and random order of the days in the greedy phases), so a cap that reaches the target in a single run may do so by chance. A candidate cap is accepted only if **all** the tested seeds reach the target (worst-case criterion), instead of the best or the average run. The seeds are drawn from a fixed pool, so every candidate is evaluated on the same seeds, and the evaluation stops as soon as one seed fails (fail-fast).

Search procedure. For each η the cap is searched in four steps:

1. *Upper bound (exponential search).* Starting at a cap of 200, the cap is doubled until the criterion is met.

2. *Lower bound.* The smallest cap (1) is tested, in case it already meets the criterion.

3. *Binary search.* The smallest cap that meets the criterion is searched in the interval between the bounds, which requires only a logarithmic number of candidates instead of testing every cap in sequence.

4. *Confirmation.* The candidate is re-evaluated with more seeds and with the ILP time limit used in production. If it fails, the cap is increased by one unit until it is confirmed.

During the search each candidate was evaluated with 8 seeds and an ILP time limit of 60 s per week, so that candidates could be compared quickly. The confirmation used 20 seeds and a limit of 600 s per week.

Limitations.

- **Sufficient, not necessarily minimal.** The binary search assumes *monotonicity*: that if a cap meets the criterion, every larger cap also does. This is plausible, since the catalogue of a larger cap contains that of a smaller one, but it is not guaranteed. The ILP is solved with a 2% gap and a time limit, so a larger catalogue may yield a worse solution, and the randomness of the heuristic adds noise to each evaluation. The confirmation step protects against a cap that passed by luck, but not against a smaller valid cap that was skipped.
- **Statistical guarantee.** The cap never failed on the seeds tested, but it is not proved that it never fails.
- **Not monotonic in η.** The cap does not grow steadily with the period length (Table 1, e.g. 26 at $\eta = 3$ and 18 at $\eta = 4$). The number of patterns needed to cover the minimums depends on how the period fits the structural constraints and not only on the size of the catalogue.
- **Instance dependent.** The values were tuned for the reference instance of the companion report *Heuristic Results (3 Shifts)*. For a different workforce or different requirements, the procedure should be repeated, since the cap that preserves the minimums depends on both.

ILP construction and solution

Once the admissible sets $\{\mathcal{P}_e\}_{e \in E}$ have been constructed for all employees, the ILP is assembled and solved. The complete mathematical formulation is given in Appendix A. In summary, the model selects exactly one pattern from $\mathcal{P}_e$ and one permitted team for each employee so as to minimise a weighted combination of three penalty terms: unmet minimum staffing requirements, unmet ideal staffing targets, and excess working days relative to the per-period target τ. The solver is run with a time limit and a relative optimality gap tolerance of 2 %; if the time limit is reached before a provably optimal solution is found, the best integer-feasible solution discovered up to that point is returned.

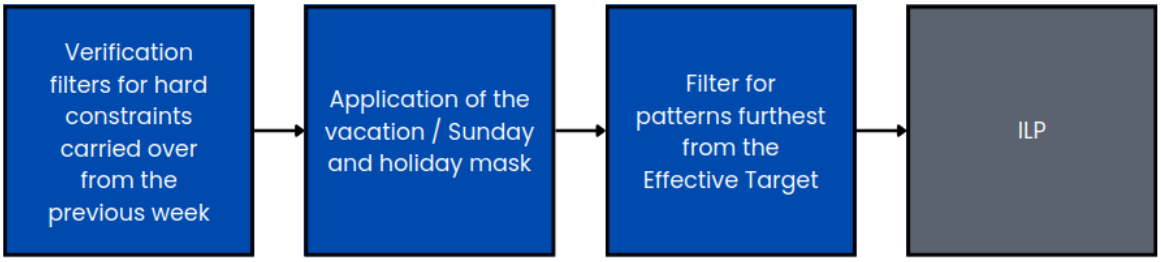


Figure 2: Sequence Steps of patterns filtered before solving

7 Pseudocode

Algorithm 1 Phase 1 — Processing weekly ILP patterns

for each $w \in W$ **do**
    Build Weekly Patterns Generation
    Pontuate_Function
    for each $e \in \tilde{E}$ **do**
        $p \leftarrow$ Patterns[e]; $t \leftarrow$ Team_Assignments[e]
        for each offset in p ($O = 0, \ldots, \eta - 1$) **do**
            $d \leftarrow d_0 + O$
            if $d \leq d_1$ **and** $p[O] \neq 0$ **then**
                Assign(e, d, $p[O]$, t)
            end if
        end for
    end for
end for

Algorithm 2 Phase 2 — Remove Excessive Ideals

for each $(\text{date}, s, t) \in Cov_{dst}$ **do**
    while $\text{cov}_{dst} > \beta_{dst}$ **do**
        Select all employees e candidates to remove in that slot
        Choose the candidate e^* with the most TotalDays_e Worked
        Remove(e^*, slot)
    end while
end for

Algorithm 3 Phase 3 — Add Sunday and Holiday Work

for phase $\in$ {0 (minimums), 1 (ideals)} **do**
    for each $d \in SH$ **do**
        Calculate necessities (M or I) to actual phase
        for each $e \in R$ **do**
            if e violates any hard constraint **then**
                continue
            end if
            for each $t \in T^*$ **do**
                Find shift $s \in S$ compatible with the transitions
                if a compatible shift s is found **then**
                    Assign(e, d, s, t)
                end if
            end for
            Calculate $Score_e$
        end for
    end for
end for

Algorithm 4 Phase 4 — Fix Ideals on Util Days

for each $(d, s, t, \gamma) \in \tilde{\mathrm{L}}$ **do**
  for each $e \in R$ **do**
    if added $\geq \gamma$ **then break**
    end if
    if α_{ed} **then continue**
    end if
    if e violates any hard constraint **then**
      continue
    end if
    ASSIGN(e, d, s, t)
    Calculate $Score_e$
  end for
end for

A Integer Linear Programming Model for Weekly Patterns

A.1 Set of Indices

Set	Description
E	Set of employees
D	Set of week days, $D = \{1, 2, \ldots, \lvert D \rvert\}$, where $\lvert D \rvert = 7$ (Note: in practice, Sundays and Holidays are pre-excluded from this set)
T	Set of teams
S	Set of possible shifts
$E_t \subseteq E$	Set of employees eligible to work in team t
$T_e \subseteq T$	Set of allowed teams for employee e

A.2 Parameters

Parameter	Description
P_e	Set of admissible weekly patterns for employee e (pre-filtered by streak, shift-transition, and holiday/vacation rules)
θ_{dst}	Minimum number of employees required on day d, shift s, team t
β_{dst}	Ideal number of employees required on day d, shift s, team t
δ_{pds}	1 if pattern p assigns shift s on day d, 0 otherwise
σ_p	Number of active worked days in pattern p
τ	Target working days per week, defined as $\tau = \frac{201}{52} \approx 3.865$
$w_{\min}$	Penalty weight for minimum shortages
w_{ideal}	Penalty weight for ideal shortages
w_{days}	Penalty weight for exceeding the weekly average target workload (τ)

A.3 Decision Variables

Variable	Description
$x_{ept} \in \{0, 1\}$	Binary decision variable; equal to 1 if employee e is assigned to weekly pattern p and team t, 0 otherwise
$u_{dst} \in \mathbb{Z}_{\geq 0}$	Integer variable representing the shortage of employees relative to the **minimum** requirement on day d, shift s, team t
$y_{dst} \in \mathbb{Z}_{\geq 0}$	Integer variable representing the shortage of employees relative to the **ideal** requirement on day d, shift s, team t
$o_e \in \mathbb{R}_{\geq 0}$	Continuous variable capturing the excess of days worked by employee e above the weekly target workload τ

A.4 Patterns Pre-Processed

Patterns follow these rules:

- Holiday, Sunday and Vacancy days are put to 0
- Patterns must respect 5 consecutive days constraint
- Patterns must respect previous shift constraint

A.5 Objective Function

Minimise the weighted sum of shortages relative to the minimum and ideal requirements over all days, shifts and teams, plus the penalty for exceeding individual weekly target working days:

$$\min \quad w_{\min} \cdot \sum_{d \in D} \sum_{s \in S} \sum_{t \in T} u_{dst} \; + \; w_{\text{ideal}} \cdot \sum_{d \in D} \sum_{s \in S} \sum_{t \in T} y_{dst} \; + \; w_{\text{days}} \cdot \sum_{e \in E} o_e \tag{A.1}$$

A.6 Coverage Expression

The number of employees from team t covering day d on shift s is given by:

$$\text{cov}_{dst} \; = \; \sum_{e \in E_t} \sum_{p \in P_e} x_{ept} \cdot \delta_{pds} \qquad d \in D,\; s \in S,\; t \in T \tag{A.2}$$

A.7 Constraints

C1 — One pattern and team per employee

Each employee must be assigned exactly one weekly pattern and team combination:

$$\sum_{p \in P_e} \sum_{t \in T_e} x_{ept} = 1 \qquad e \in E \tag{A.3}$$

C2 — Weekly workload pacing

Measures the absolute excess of days worked by each employee over the weekly target average τ. Together with the objective function, this forces o_e to capture the workload surplus:

$$\sum_{p \in P_e} \sum_{t \in T_e} \sigma_p \cdot x_{ept} - \tau \; \leq \; o_e \qquad e \in E \tag{A.4}$$

C3 — Minimum coverage requirements

These constraints (together with the objective function) guarantee that variable u_{dst} has the shortage of employees for each day d, shift s and team t.

$$\text{cov}_{dst} + u_{dst} \; \geq \; \theta_{dst} \qquad d \in D,\; s \in S,\; t \in T \tag{A.5}$$

C4 — Ideal coverage requirements

Analogously, y_{dst} captures the deficit relative to the ideal target:

$$\mathrm{cov}_{dst} + y_{dst} \;\geq\; \beta_{dst} \qquad d \in D,\; s \in S,\; t \in T \tag{A.6}$$